Database Management Systems

Course Code: CT-261

Complex Computing Problem

Instructor: Dr. Raheela Asif

Football Transfer Management System

A DBMS Solution

Section C — Batch 2024

Team Participation & Work Distribution

Football Transfer Management System — CT-261

Name	Roll No.	Tasks / Contributions
Haris Zamir	CT-24125	EER Model (Specialization/Generalization, Aggregation, EER diagram), Physical Model (ER diagram), Documentation. **(II-C, II-D)**
Muhammad Raza	CT-24138	Logical Model (ER diagram, Cardinalities allocation), Normalization (1NF to BCNF). **(II-B, IV)**
Muhammad Rafay	CT-24141	Physical Model (DDL scripting, Constraint definition), SQL Query generation (1 to 6). **(II-C, V)**
Muhammad Nizam	CT-24147	Conceptual Model (ER diagram, Relationships definition), Functional dependencies, Business rules definition. **(I, II-A, III)**

I. Organization — Problem Statement

Business Context

Football clubs, agents, and governing leagues require a centralized database system to manage player transfers, contracts, and financial transactions. Currently, most mid-level clubs rely on fragmented spreadsheets and manual record-keeping, which leads to data inconsistency, missed transfer deadlines, and financial miscalculations.

Business Requirements

- Track all player information including market value, nationality, and position.
- Record transfer dealings between clubs including fees, dates, and type (loan/permanent).
- Manage player contracts with salary, start date, and end date.
- Register agents and track commission rates and contact information.
- Associate clubs to leagues and support multi-league environments.
- Maintain a Person supertype to store shared personal information across Players and Agents.

Limitations of Current System

- No referential integrity — player records can be deleted without removing associated contracts or transfers.
- Duplicate data entry across spreadsheets causes inconsistency.
- Inability to run complex analytical queries across entities.
- No audit trail for contract history or transfer negotiations.
- No separation between personal identity data and role-specific data (e.g. Player vs Agent).

II-A. Conceptual Model

The conceptual ER diagram models seven core entities: Person, Player, Agent, Club, Contract, Transfer, and League. Person is the supertype for both Player and Agent, connected via IsA relationships. The diagram shows entities, relationships, and primary key attributes. No cardinalities or data types are shown at the conceptual level.

Entity	Primary Key	Attributes	Description
Person	personId {PK}	name, dateOfBirth, nationality, contactInfo	Stores shared personal information for Players and Agents.
Player	playerId {PK}	personId, position, marketValue	Represents a football player in the system.
Agent	agentId {PK}	personId, commissionRate	Represents an agent who manages player transfers.

Entity	Primary Key	Attributes	Description
Club	clubId {PK}	name, country, budget	A football club that signs players and participates in transfers.
Contract	contractId {PK}	salary, startDate, endDate	A contract between a player and a club.
Transfer	transferId {PK}	fee, transferDate, transferType	A transfer record documenting the movement of a player between clubs.
League	leagueId {PK}	name, country	A football league that clubs compete in.

Relationships

The following relationships exist between entities. Cardinalities are not shown at the conceptual level.

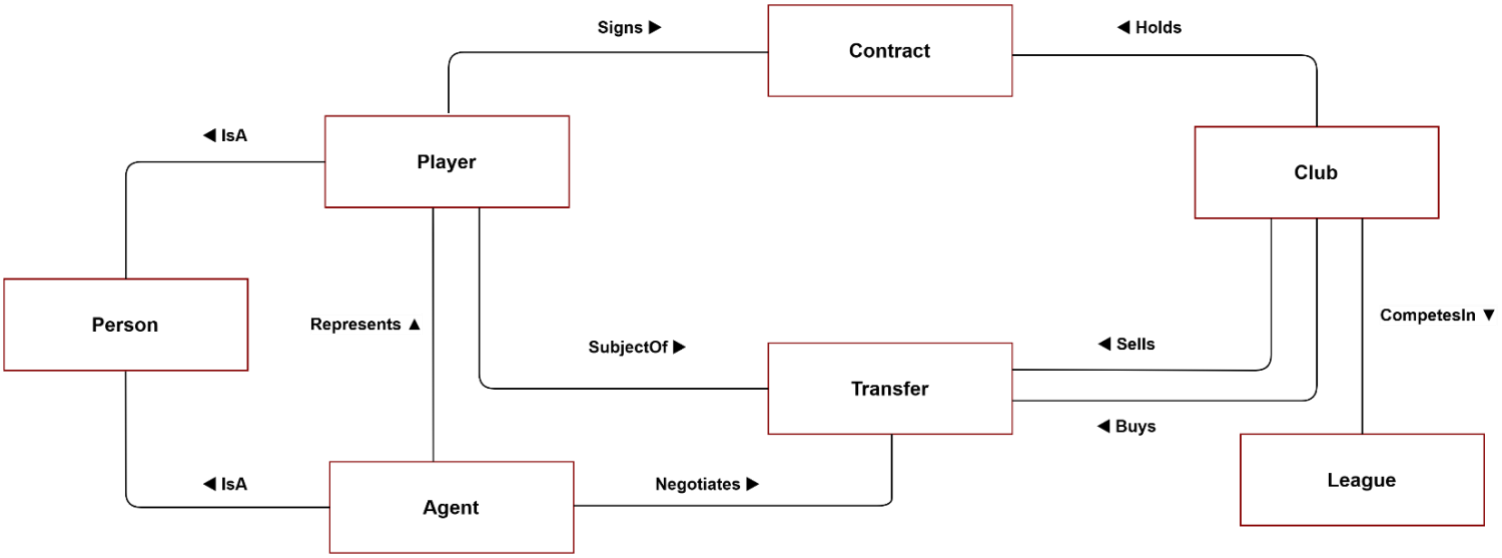
Relationship	Entities Involved	Description
Signs	Player — Contract	A player signs a contract with a club.
Holds	Club — Contract	A club holds contracts for its players.
Sells	Club — Transfer	A club sells a player through a transfer.
Buys	Club — Transfer	A club buys a player through a transfer.
SubjectOf	Player — Transfer	A player is the subject of a transfer record.
Negotiates	Agent — Transfer	An agent negotiates a transfer on behalf of a player.
CompetesIn	Club — League	A club competes in a football league.
Represents	Agent — Player	An agent represents a player. Note: this relationship is present in the ER diagram but is removed in the EER diagram, where the shared Person supertype and formal specialization make it redundant.
IsA	Player — Person	A player is a specialization of a person.
IsA	Agent — Person	An agent is a specialization of a person.

Business Rules

- A Transfer must have exactly one buying club and at most one selling club (free agent signings have no seller).
- A Contract must be linked to both a Player and a Club.
- Every Person must be either a Player or an Agent — a Person cannot exist without a role.
- An Agent cannot also be a Player and vice versa (disjoint specialization).
- Budget and salary are currency fields; transferDate and dateOfBirth are date fields.

Conceptual ER Diagram

Shows all seven entities with their primary keys and key attributes, connected by labeled relationship lines. No cardinalities or data types. Person is the supertype connected to Player and Agent via IsA lines. Club connects to League via CompetesIn. Transfer is connected to Player (SubjectOf), Agent (Negotiates), and Club (Sells/Buys).



II-B. Logical Model — Relational Schema

The logical model translates the conceptual diagram into a full relational schema. Foreign keys are introduced at this level to enforce referential integrity. Cardinalities are also defined here to specify the exact participation rules between entities.

Relational Tables

Table	Primary Key	Columns
Person	personId {PK}	name, dateOfBirth, nationality, contactInfo
Player	playerId {PK}	personId {FK->Person}, position, marketValue, agentId {FK->Agent}
Agent	agentId {PK}	personId {FK->Person}, commissionRate
Club	clubId {PK}	name, country, budget, leagueId {FK->League}
Contract	contractId {PK}	salary, startDate, endDate, playerId {FK->Player}, clubId {FK->Club}
Transfer	transferId {PK}	fee, transferDate, transferType, playerId {FK->Player}, sellingClubId {FK->Club NULL}, buyingClubId {FK->Club}, agentId {FK->Agent}
League	leagueId {PK}	name, country

Relationships & Cardinalities

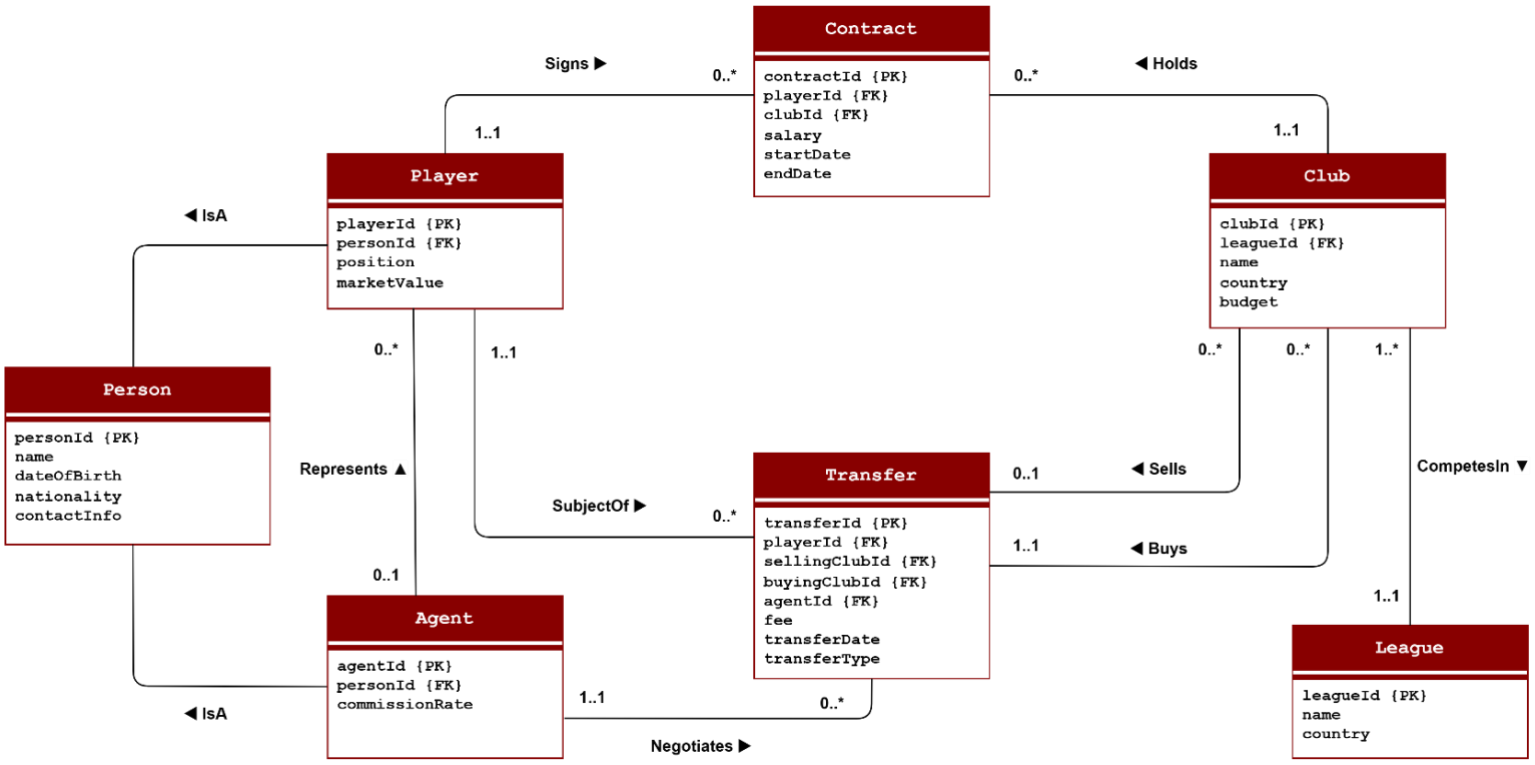
The following table defines the cardinality of each relationship, added at the logical level:

Relationship	Entities Involved	Cardinality	Description
Signs	Player — Contract	1..1 : 0..*	Each contract is signed by exactly one player; a player may sign many contracts.
Holds	Club — Contract	1..1 : 0..*	A club holds many contracts; each contract belongs to exactly one club.
Sells	Club — Transfer	0..* : 0..1	A club may sell in zero or many transfers; a transfer has at most one selling club.
Buys	Club — Transfer	0..* : 1..1	A club may buy in zero or many transfers; every transfer has exactly one buying club.
SubjectOf	Player — Transfer	1..1 : 0..*	A player may be the subject of many transfers; each transfer involves exactly one player.

Relationship	Entities Involved	Cardinality	Description
Negotiates	Agent — Transfer	1..1 : 0..*	An agent may negotiate many transfers; each transfer is handled by exactly one agent.
CompetesIn	Club — League	1..* : 1..1	A league has one or more clubs; each club competes in exactly one league.
Represents	Agent — Player	0..1 : 0..*	An agent may represent many players; a player may optionally have an agent. Removed in EER diagram as the Person specialization supersedes this direct relationship.

Logical ER Diagram

Shows all seven tables with primary keys, non-key attributes, foreign keys, and relationship lines with cardinality labels (e.g. 1..1, 0..). IsA lines connect Person to Player and Agent. No data types shown. CompetesIn connects Club to League with 1..* and 1..1.*



II-C. Physical Design — DDL Scripts

The physical design builds on the logical model by adding full data types, NOT NULL constraints, AUTO_INCREMENT on all integer primary keys, and explicit FOREIGN KEY declarations. The schema below is written in standard SQL and can be executed directly to create the database.

Design Constraints

The database is designed with multiple tables and records. The following field types satisfy the requirement for at least two distinct field types:

- Numeric — INT (all primary and foreign keys), DECIMAL (marketValue, budget, commissionRate).
- Alpha — VARCHAR (name, position, nationality, country, transferType, contactInfo).
- Date/Time — DATE (dateOfBirth, startDate, endDate, transferDate).
- Currency — DECIMAL(15,2) (fee, budget, salary, marketValue).

Physical ER Diagram

Mirrors the logical diagram with full data types (INT, VARCHAR, DECIMAL, DATE) annotated beside each attribute. Foreign keys are highlighted and labeled FK -> TableName. UNIQUE constraints on personId in Player and Agent enforce the one-to-one IsA relationship.

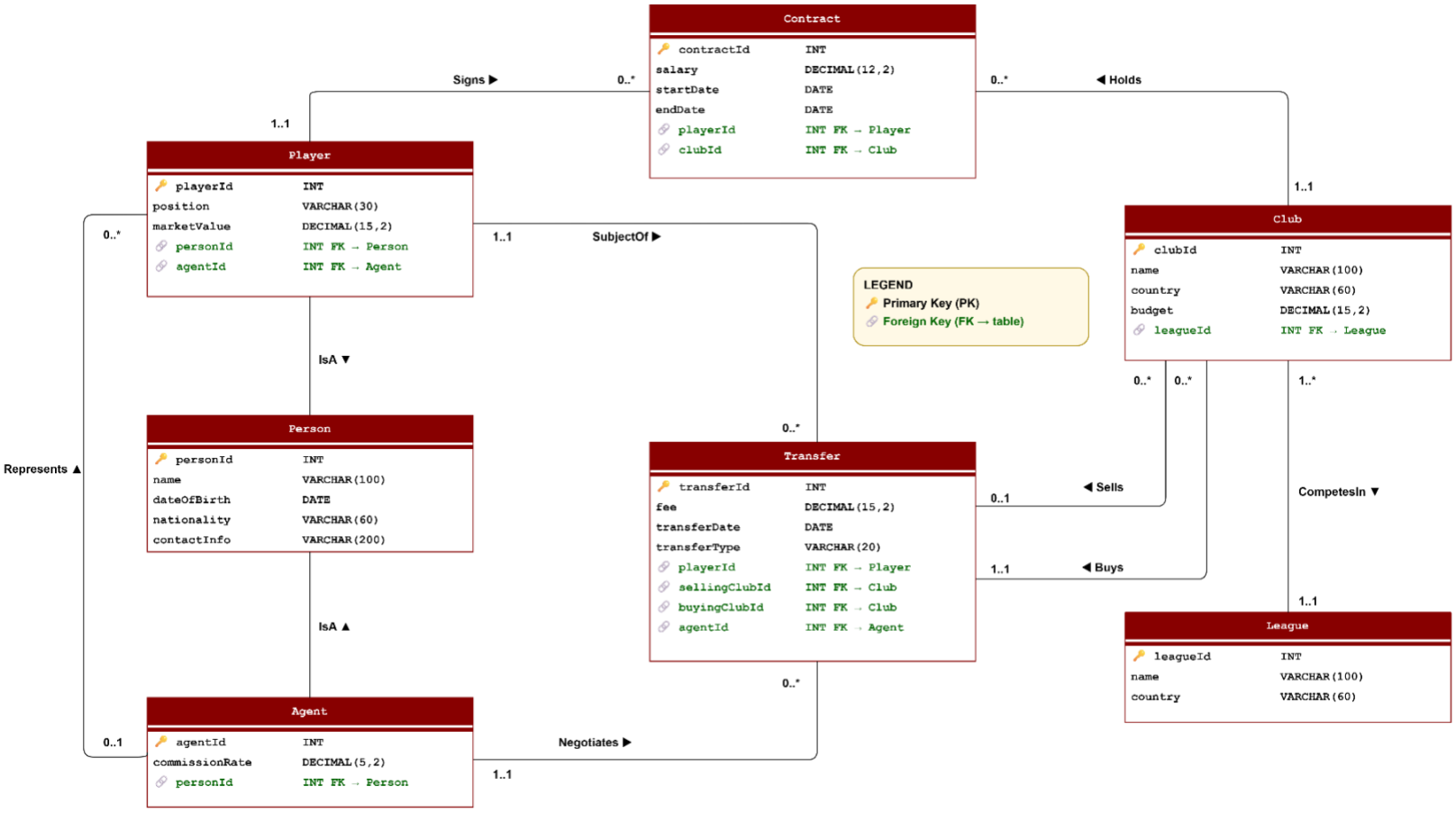


Table Definitions

The following SQL DDL statements define all seven tables with their data types, primary keys, foreign keys, and constraints:

```
-- League table (no dependencies, created first)
CREATE TABLE League (
  leagueId    INT           PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(100)  NOT NULL,
  country     VARCHAR(60)   NOT NULL
);
```

```
-- Person supertype (shared identity for Players and Agents)
CREATE TABLE Person (
  personId    INT           PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(100)  NOT NULL,
  dateOfBirth DATE          NOT NULL,
  nationality VARCHAR(60)   NOT NULL,
  contactInfo VARCHAR(200)
);
```

```
-- Agent subtype of Person
CREATE TABLE Agent (
  agentId        INT          PRIMARY KEY AUTO_INCREMENT,
  personId       INT          NOT NULL UNIQUE,
  commissionRate DECIMAL(5,2) NOT NULL,
  FOREIGN KEY (personId) REFERENCES Person(personId)
);
```

```
-- Club (belongs to a League)
CREATE TABLE Club (
  clubId    INT           PRIMARY KEY AUTO_INCREMENT,
  name      VARCHAR(100)  NOT NULL,
  country   VARCHAR(60)   NOT NULL,
  budget    DECIMAL(15,2) NOT NULL,
  leagueId  INT           NOT NULL,
  FOREIGN KEY (leagueId) REFERENCES League(leagueId)
);
```

```
-- Player subtype of Person (optionally represented by an Agent)
CREATE TABLE Player (
  playerId    INT           PRIMARY KEY AUTO_INCREMENT,
  personId    INT           NOT NULL UNIQUE,
  position    VARCHAR(30)   NOT NULL,
  marketValue DECIMAL(15,2),
  agentId     INT,
  FOREIGN KEY (personId) REFERENCES Person(personId),
  FOREIGN KEY (agentId)  REFERENCES Agent(agentId)
);
```

```
-- Contract (links a Player to a Club)
CREATE TABLE Contract (
  contractId INT           PRIMARY KEY AUTO_INCREMENT,
  salary     DECIMAL(12,2) NOT NULL,
  startDate  DATE          NOT NULL,
  endDate    DATE          NOT NULL,
  playerId   INT           NOT NULL,
  clubId     INT           NOT NULL,
  FOREIGN KEY (playerId) REFERENCES Player(playerId),
  FOREIGN KEY (clubId)   REFERENCES Club(clubId)
);
```

```
-- Transfer (records movement of a player between clubs)
CREATE TABLE Transfer (
  transferId    INT           PRIMARY KEY AUTO_INCREMENT,
  fee           DECIMAL(15,2) NOT NULL,
  transferDate  DATE          NOT NULL,
  transferType  VARCHAR(20)   NOT NULL,  -- 'Permanent' or 'Loan'
  playerId      INT           NOT NULL,
  sellingClubId INT,                     -- NULL = free agent signing
  buyingClubId  INT           NOT NULL,
  agentId       INT           NOT NULL,
  FOREIGN KEY (playerId)      REFERENCES Player(playerId),
  FOREIGN KEY (sellingClubId) REFERENCES Club(clubId),
  FOREIGN KEY (buyingClubId)  REFERENCES Club(clubId),
  FOREIGN KEY (agentId)       REFERENCES Agent(agentId)
);
```

II-D. Enhanced ER Diagram (EER)

The EER diagram extends the conceptual ER diagram with two additional modeling concepts:

Specialization / Generalization

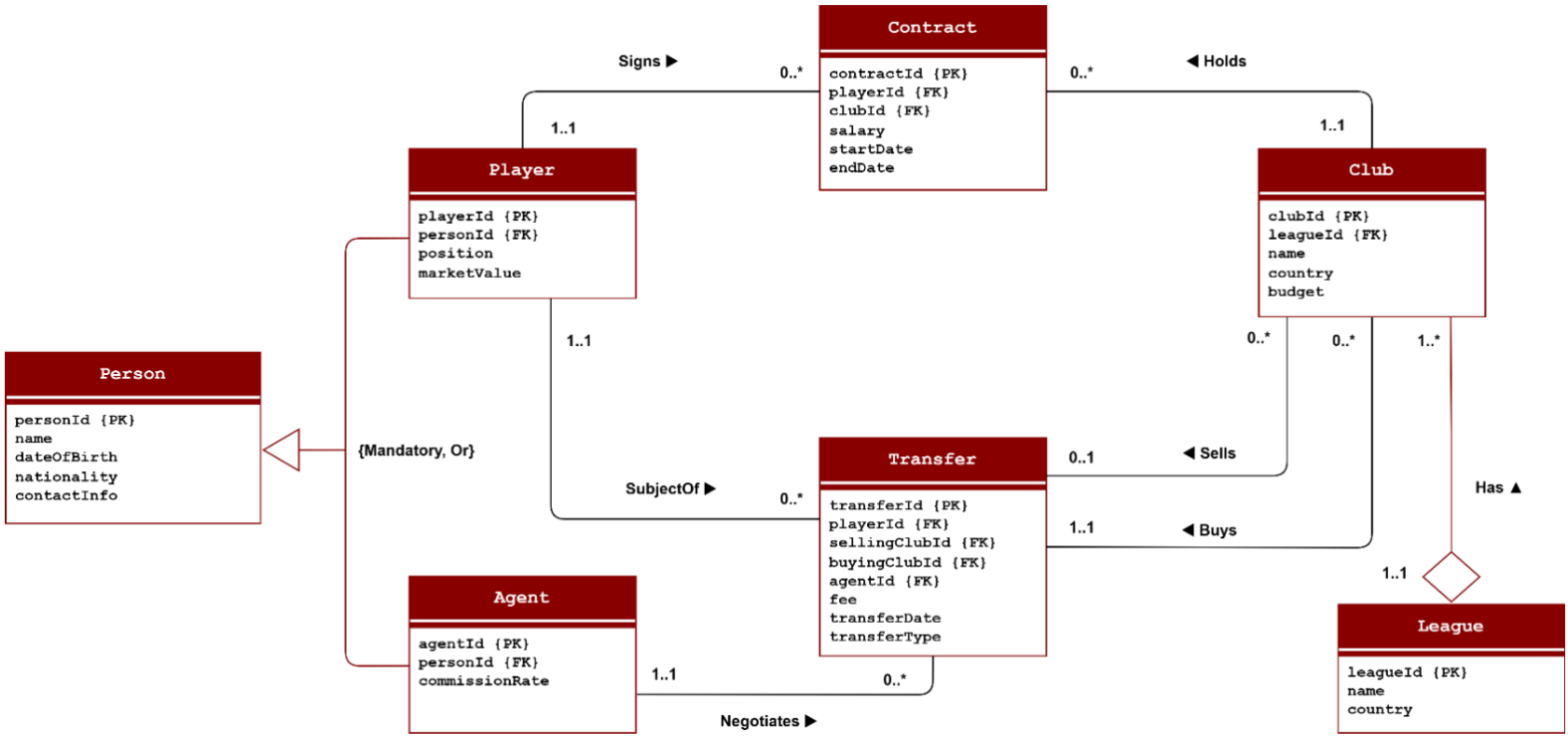
Person is the superclass, with Player and Agent as subclasses. A hollow triangle pointing toward Person represents this hierarchy, with {Mandatory, Or} placed beside it. Mandatory means every Person must belong to at least one subclass. Or (disjoint) means a Person cannot be both a Player and an Agent. The Represents relationship present in the ER diagram is removed in the EER diagram, as the shared Person supertype makes it redundant.

Aggregation

League Has Club is modeled as an aggregation. An open hollow diamond is placed on the League end of the relationship line, indicating League is the whole entity and Club is the part. The relationship is renamed from CompetesIn (ER diagram) to Has (EER diagram) to reflect whole-part ownership semantics. The arrow direction also changes: in the ER diagram it reads Club → League (CompetesIn), while in the EER diagram it reads League → Club (Has). Cardinalities remain 1..1 near Club and 1..* near League.

EER Diagram

Extends the ER diagram with: (1) Specialization — hollow triangle with {Mandatory, Or} between Person and subclasses Player and Agent; Represents relationship removed. (2) Aggregation — hollow diamond on League replacing CompetesIn with Has, arrow direction reversed. All other relationships unchanged.



III. Functional Dependencies

The following functional dependencies define which attributes are determined by the primary key of each table:

Person

```
personId -> name, dateOfBirth, nationality, contactInfo
```

Player

```
playerId -> personId, position, marketValue, agentId
```

Agent

```
agentId -> personId, commissionRate
```

Club

```
clubId -> name, country, budget, leagueId
```

Contract

```
contractId -> salary, startDate, endDate, playerId, clubId
```

Transfer

```
transferId -> fee, transferDate, transferType, playerId, sellingClubId,
buyingClubId, agentId
```

League

```
leagueId -> name, country
```

IV. Normalization (1NF → BCNF)

The Transfer record is selected as the main example because it contains attributes related to multiple entities such as Player, Club, Agent, and Contract information. The normalization process below demonstrates how the relation evolves from 1NF to BCNF.

Unnormalized Form (UNF)

One row:

transferId	fee	transferDate	transferType	playerName	playerNationality	playerDateOfBirth
1001	80000000	2024-07-01	Permanent	Kylian Mbappe	France	1998-12-20

sellingClubName	sellingClubCountry	buyingClubName	buyingClubCountry	agentName	agentCommissionRate	salary	contractStartDate	contractEndDate
PSG	France	Real Madrid	Spain	Jorge Mendes	10%	25000000	2024-07-01	2029-06-30

Problems in UNF:

1. Player details are repeated in every transfer.
2. Club details are duplicated for buying and selling clubs.
3. Agent information repeats across multiple transfers.
4. Contract details are embedded inside transfer records.
5. Update anomalies and insertion anomalies may occur.

First Normal Form (1NF)

A relation is in 1NF when:

1. All attributes contain atomic values.
2. No repeating groups exist.
3. Each record has a unique primary key.

In this stage, IDs are introduced to uniquely identify related entities, One row:

transferId	fee	transferDate	transferType	playerId	playerName	playerNationality
1001	80000000	2024-07-01	Permanent	501	Kylian Mbappe	France

sellingClubId	sellingClubName	buyingClubId	buyingClubName	agentId	agentName	salary	contractStartDate	contractEndDate
301	PSG	302	Real Madrid	201	Jorge Mendes	25000000	2024-07-01	2029-06-30

Problems in 1NF:

Improvements Achieved in 1NF

1. All values are atomic.
2. transferId uniquely identifies each row.

Remaining Problems

1. playerName and playerNationality depend on playerId, not transferId.
2. sellingClubName depends on sellingClubId.
3. buyingClubName depends on buyingClubId.
4. agentName depends on agentId.

These are partial and transitive dependencies.

Second Normal Form (2NF)

A relation is in 2NF when:
1. It is already in 1NF.
2. No non-key attribute depends partially on another non-key attribute.

To remove partial dependencies, entity-specific data is separated into independent tables.

Player Table

playerId	playerName	playerNationality
501	Kylian Mbappe	France

Agent Table

agentId	agentName	commissionRate
201	Jorge Mendes	10%

Club Table

clubId	clubName	clubCountry
301	PSG	France
302	Real Madrid	Spain

Transfer Table:

transferId	fee	transferDate	transferType	playerId	sellingClubId	buyingClubId	agentId	salary	contractStartDate	contractEndDate
1001	80000000	2024-07-01	Permanent	501	301	302	201	25000000	2024-07-01	2029-06-30

Improvements Achieved in 2NF

1. Player information stored separately.

2. Club information stored separately.
3. Agent information stored separately.
4. Redundant repeating entity data removed.

Remaining Problems

1. salary, contractStartDate, and contractEndDate are contract-related attributes.
2. These attributes do not directly describe the transfer itself.
3. A transitive dependency still exists because contract information belongs to a separate entity.

Third Normalized Form (3NF)

A relation is in 3NF when:

1. It is already in 2NF.
2. No transitive dependencies exist.

Contract information is separated into its own relation.

Contract Table

contractId	salary	startDate	endDate	playerId	clubId
701	25000000	2024-07-01	2029-06-30	501	302

Transfer Table

transferId	fee	transferDate	transferType	playerId	sellingClubId	buyingClubId	agentId
1001	80000000	2024-07-01	Permanent	501	301	302	201

Improvements Achieved in 3NF

1. Contract information isolated into its own table.
2. No transitive dependencies remain.
3. Every non-key attribute depends directly on the primary key.

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if:

1. For every functional dependency X → Y, X must be a superkey.

The final decomposed relations are checked below.

Player Table

playerId	playerName	playerNationality
501	Kylian Mbappe	France

Functional Dependency:

1. playerId → playerName, playerNationality
2. playerId is the primary key.
3. BCNF satisfied.

Agent Table

agentId	agentName	commissionRate
201	Jorge Mendes	10%
clubId	clubName	clubCountry
301	PSG	France
302	Real Madrid	Spain

Functional Dependency:

1. agentId → agentName, commissionRate
2. agentId is the primary key.
3. BCNF satisfied.

Club Table

clubId	clubName	clubCountry
301	PSG	France
302	Real Madrid	Spain

Functional Dependency:

1. clubId → clubName, clubCountry
2. clubId is the primary key.
3. BCNF satisfied.

Contract Table

contractId	salary	startDate	endDate	playerId	clubId
701	25000000	2024-07-01	2029-06-30	501	302

Functional Dependency:

1. contractId → salary, startDate, endDate, playerId, clubId
2. contractId is the primary key.
3. BCNF satisfied.

Transfer Table

transferId	fee	transferDate	transferType	playerId	sellingClubId	buyingClubId	agentId
1001	80000000	2024-07-01	Permanent	501	301	302	201

Functional Dependency:

1. transferId → fee, transferDate, transferType, playerId, sellingClubId, buyingClubId, agentId

2. transferId is the primary key.
3. BCNF satisfied.

Additional BCNF Verification

A relation is in BCNF if for every functional dependency X → Y, X is a superkey. Reviewing all tables:

1. Person: personId → all attributes. Only determinant is the PK — BCNF satisfied.
2. Player: playerId → all attributes. No other determinant — BCNF satisfied.
3. Agent: agentId → all attributes. No other determinant — BCNF satisfied.
4. Club: clubId → all attributes. No other determinant — BCNF satisfied.
5. Contract: contractId → all attributes — BCNF satisfied.
6. Transfer: transferId → all attributes — BCNF satisfied.
7. League: leagueId → all attributes — BCNF satisfied.

All tables are in BCNF. No further decomposition required.

1. Person Table

Primary Key: personId

Functional Dependencies

- personId → fullName, dateOfBirth, nationality, contactInfo, ...

The attribute personId uniquely identifies each record in the relation and determines all other non-key attributes.

BCNF Evaluation

Since the only determinant in the table is the primary key (personId), and a primary key is by definition a superkey, the relation satisfies the BCNF condition.

Conclusion

The Person table is in BCNF because all functional dependencies have a superkey determinant.

2. Player Table

Primary Key: playerId

Functional Dependencies

- playerId → position, marketValue, clubId, personId, ...

Each player is uniquely identified by playerId, which determines all remaining attributes in the table.

BCNF Evaluation

No partial, transitive, or non-key dependencies were identified. The determinant (playerId) is a superkey.

Conclusion

The Player table satisfies BCNF requirements.

3. Agent Table

Primary Key: agentId

Functional Dependencies

- agentId → agencyName, licenseNumber, personId, ...

The agentId uniquely identifies every agent record and functionally determines all associated attributes.

BCNF Evaluation

There are no additional determinants apart from the primary key. Therefore, every functional dependency has a superkey on the left-hand side.

Conclusion

The Agent table is in BCNF.

4. Club Table

Primary Key: clubId

Functional Dependencies

- clubId → clubName, country, foundedYear, leagueId, ...

Each club record is uniquely identified by clubId.

BCNF Evaluation

The only determinant is the primary key, which is a superkey. No anomalies or dependency violations were identified.

Conclusion

The Club table satisfies BCNF.

5. Contract Table

Primary Key: contractId

Functional Dependencies

contractId → playerId, clubId, salary, startDate, endDate, ...

The contractId uniquely determines all contract-related information.

BCNF Evaluation

No non-key determinant exists in the relation. Since the determinant is the primary key, the relation conforms to BCNF.

Conclusion

The Contract table is in BCNF.

6. Transfer Table

Primary Key: transferId

Functional Dependencies

- transferId → playerId, fromClubId, toClubId, transferFee, transferDate, ...

The transferId uniquely identifies each transfer transaction.

BCNF Evaluation

All dependencies are fully dependent on the primary key, and no additional functional dependencies violate BCNF conditions.

Conclusion

The Transfer table satisfies BCNF.

7. League Table

Primary Key: leagueId

Functional Dependencies

- leagueId → leagueName, country, governingBody, ...

Each league record is uniquely identified by leagueId.

BCNF Evaluation

The only determinant in the table is the primary key, which is a superkey.

Conclusion

The League table is in BCNF.

V. Sample SQL Queries

Query 1 — List all players with their agent and market value

```
SELECT p.playerId, per.name AS playerName, p.position,
       p.marketValue, per2.name AS agentName, a.commissionRate
FROM   Player p
JOIN   Person per   ON p.personId  = per.personId
LEFT JOIN Agent a   ON p.agentId   = a.agentId
LEFT JOIN Person per2 ON a.personId = per2.personId
ORDER BY p.marketValue DESC;
```

Query 2 — Total spending per club in a given year

```
SELECT c.name AS club, SUM(t.fee) AS totalSpent
FROM   Transfer t
JOIN   Club c ON t.buyingClubId = c.clubId
WHERE  YEAR(t.transferDate) = 2024
GROUP BY c.name
ORDER BY totalSpent DESC;
```

Query 3 — Active contracts expiring within 6 months

```
SELECT per.name AS player, c.name AS club,
       con.salary, con.endDate
FROM   Contract con
JOIN   Player p   ON con.playerId = p.playerId
JOIN   Person per ON p.personId   = per.personId
JOIN   Club c     ON con.clubId   = c.clubId
WHERE  con.endDate BETWEEN CURDATE()
       AND DATE_ADD(CURDATE(), INTERVAL 6 MONTH);
```

Query 4 — Players transferred more than once

```
SELECT per.name, COUNT(t.transferId) AS numTransfers
FROM   Transfer t
JOIN   Player p   ON t.playerId  = p.playerId
JOIN   Person per ON p.personId  = per.personId
GROUP BY p.playerId, per.name
HAVING COUNT(t.transferId) > 1
ORDER BY numTransfers DESC;
```

Query 5 — Clubs and the league they belong to (with budget)

```
SELECT l.name AS league, c.name AS club,
       c.country, c.budget
FROM   Club c
JOIN   League l ON c.leagueId = l.leagueId
```

```
ORDER BY l.name, c.budget DESC;
```

Query 6 — Agent commission earned per transfer

```
SELECT per.name AS agent, t.transferId,
       t.fee, (t.fee * a.commissionRate / 100) AS commission
FROM   Transfer t
JOIN   Agent a    ON t.agentId   = a.agentId
JOIN   Person per ON a.personId  = per.personId
ORDER BY commission DESC;
```